

You are here: [Home](#) » [Cisco](#) » [CCNA 200-301](#)

Introduction to TCP and UDP

In this lesson we are going to take a look at our transport protocols TCP and UDP. If you know about IP and IP packets you know that we require a transport protocol to send our IP packets.

I want to focus on the transport protocols that are used most of the time:

- **TCP (Transmission Control Protocol)**
- **UDP (User Datagram Protocol)**

So why do we have 2 different transport protocols here, why do we care and when do we need one over another?

The short answer is:

- TCP is a **reliable** protocol.
- UDP is a **unreliable** or **best-effort** protocol.

Unreliable you might think? Why do I want data transport which is unreliable? Does that make any sense? Let me tell you a little story to explain the difference between the two protocols.

You are sitting behind your computer and downloading the latest greatest movie in 1080P HD with 7.1 surround super sound directly from Universal studio's brand new "download on demand" service (hey you never know...it might happen one day...). This file is 20GB and after downloading 10GB there's something going wrong and a couple of IP packets don't make it to your computer, as soon as the entire download is done you try to play the movie and you get all kind of errors. Unable to watch the movie you are frustrated and head for the local dvd rental place to watch some low-quality movie...

Ok maybe I exaggerate a bit but I think you get the idea; you want to make sure the transport of your download to your computer is **reliable** which is why we use TCP. In case some of the IP packets don't make it to your computer you want to make sure this data will be retransmitted to your computer!

In our second story you are the network engineer for a major company and you just told your boss how awesome this brand new open source Voice over IP solution is. You decide to implement this new VoIP solution and to get rid of all the analog phones but your users are now complaining big time that their phone call quality is horrible. You contact the open source VoIP solution provider and you find out that they thought it would be a good idea to use a **reliable** transport protocol like TCP since well, we want phone calls to be reliable right?



Course Contents

CCNA 200-301

- + Unit 1: Introduction
 - Unit 2: Network Fundamentals
 - [Introduction to the OSI Model](#)
 - [Introduction to IPv4 \(Internet Protocol\)](#)
 - [IPv4 Packet Header](#)
 - [Address Resolution Protocol \(ARP\)](#)
 - [Introduction to TCP and UDP](#)**
 - [TCP Header](#)
 - [TCP Window Size Scaling](#)
 - [Introduction to ICMP](#)
 - [Introduction to DNS](#)
 - [Introduction to Cisco Command Line Interface \(CLI\)](#)
 - [User mode and Privileged mode](#)
 - + Unit 3: Network Access
 - + Unit 4: IP Connectivity
 - + Unit 5: IP Services
 - + Unit 6: IPv6
 - + Unit 7: Security Fundamentals
 - + Unit 8: Network Management
 - + Unit 9: Network Design
 - + Unit 10: Automation and Programmability
 - + Unit 11: Cloud Computing
 - + Unit 12: Practice Exam

Wrong thinking! TCP does error correction which means that data that didn't make it to your computer will be retransmitted. How weird will your phone call sound if you are talking to someone and you hear something that they said a few seconds ago? It's real-time so we don't want retransmission. It's better to send VoIP packets and lose a few than retransmitting them afterwards, your VoIP codec can also fix packet loss up to a certain degree. In this example we'll want to use a **best effort** or **unreliable** protocol which is UDP.

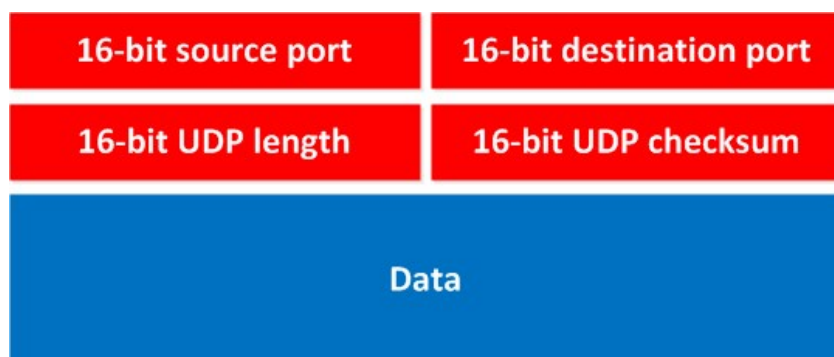
	TCP	UDP
Connection Type:	Connection-oriented	Connectionless
Sequencing:	Yes	No
Usage:	Downloads	VoIP
	File Sharing	Video (streaming)
	Printing	

What do we have in the table above? First of all you see "connection type". TCP is **connection-oriented** which means it will **"setup" a connection** and then start transferring data. UDP is connectionless which means it will just start sending and doesn't care if it arrives yes or not. The connection that TCP will setup is called the **"3 way handshake"** which I will show you in a minute.

Sequencing means that we use a **sequence number**, if you download a big file you need to make sure that you can put all those packets back in the right order. As you can see UDP does not offer this feature, there's no sequence number there.

So what about VoIP? Don't we need to put those packets back in order at the receiver side? Well actually yes we do otherwise we get some strange conversations. UDP does not offer this "sequencing" feature though...let me tell you a little secret: for VoIP it's not just UDP that we use but we also use RTP which does offer sequencing! (And some other cool features we need for VoIP).

Let's take a look at an UDP header:



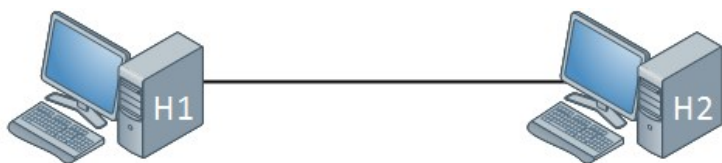
You can see how simple it is, it has the source and destination port number (this is how we know for which application the data is meant), there's a checksum and the length.

Let's sum up what we now know about UDP:

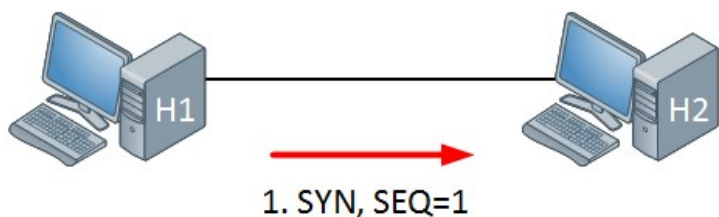
- It operates on the transport layer of the OSI model.
- Is a connectionless protocol, does not setup a connection...just sends data.
- Limited error correction because we have a checksum.

- Best-effort or unreliable protocol.
- No data-recovery features.

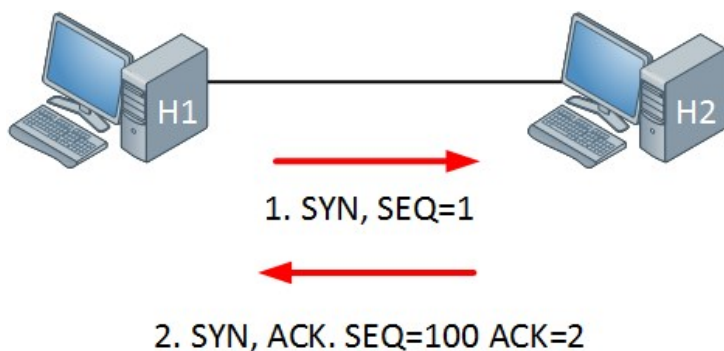
Now let's see what TCP can offer us. First of all since TCP is a reliable protocol it will "setup" a connection before we start sending any data. This connection is called the "**3 way handshake**". Let me explain this by showing two computers that want to send data to each other in a reliable way:



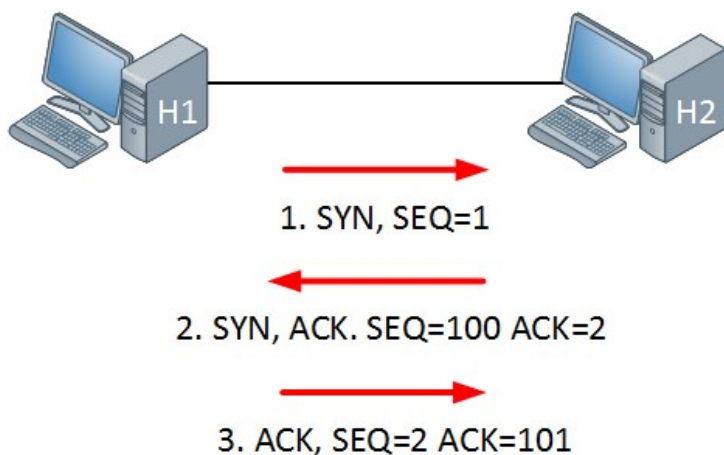
H1 wants to send data to H2 in a reliable way, so we are going to use TCP to accomplish this. First we will setup the connection by using a 3-way handshake, let me walk you through the process:



First our H1 will send a **TCP SYN**, telling H2 that it wants to setup a connection. There's also a sequence number and to keep things simple I picked number 1.



H2 will respond to H1 by sending a **SYN,ACK** message back. You can see it picks its own sequence number 100 (I just picked a random number) and it sends ACK=2. ACK=2 means that it acknowledges that it has received the TCP SYN from H1 which had sequence number 1 and that it is ready for the next message with sequence number 2.



The last step is that H1 will send an **acknowledgement** towards H2 in response of the SYN that H2 sent towards H1. You can see it sends ACK=101 which means it acknowledges the SEQ=100 from H2. Since H2 sent a ACK=2 towards H1, H1 now knows it can send the next message with sequence number 2.

To simplify things a little bit, it looks like this:

- H1 sends a **TCP SYN**. (I want to talk to you)
- H2 sends a **TCP SYN,ACK**. (I accept that you want to talk to me, and I want to talk to you as well)
- H1 sends a **TCP ACK**. (I accept that you want to talk to me)

Let me show you an example in Wireshark what this looks like on a real network:

10.000000	192.168.1.2	174.143.213.184	TCP	54841 > http [SYN] Seq=0 Win=5840 L
20.046770	174.143.213.184	192.168.1.2	TCP	http > 54841 [SYN, ACK] Seq=0 Ack=1
30.046803	192.168.1.2	174.143.213.184	TCP	54841 > http [ACK] Seq=1 Ack=1 Win=

In this example computer with IP address 192.168.1.2 wants to setup a connection with 174.143.213.184 and it's sending a TCP SYN.

174.143.213.184 is responding by sending a TCP SYN,ACK in return.

Finally 192.168.1.2 sends a TCP ACK to finish the 3 way handshake.

Let's see those packets in detail, first we look at the TCP SYN:

```
▼ Transmission Control Protocol, Src Port: 54841 (54841), Dst Port: http (80), Seq: 0, Len: 0
  Source port: 54841 (54841)
  Destination port: http (80)
  [Stream index: 0]
  Sequence number: 0 (relative sequence number)
  Header length: 40 bytes
  ► Flags: 0x02 (SYN)
    Window size: 5840
  ► Checksum: 0x85f0 [validation disabled]
  ► Options: (20 bytes)
```

You can see in the “Flags” section that the SYN-bit has been set. On the top right you can see “Seq: 0” which is the sequence number.

```
▼ Transmission Control Protocol, Src Port: http (80), Dst Port: 54841 (54841), Seq: 0, Ack: 1, Len: 0
  Source port: http (80)
  Destination port: 54841 (54841)
  [Stream index: 0]
  Sequence number: 0 (relative sequence number)
  Acknowledgement number: 1 (relative ack number)
  Header length: 40 bytes
  ► Flags: 0x12 (SYN, ACK)
```

In this example you see that in the “Flags” section both the SYN and ACK bit are set, also on the top you can see “Seq :0” and “Ack:1”. This computer is acknowledging the SYN-bit from the other computer.

```
▼ Transmission Control Protocol, Src Port: 54841 (54841), Dst Port: http (80), Seq: 1, Ack: 1, Len: 0
  Source port: 54841 (54841)
  Destination port: http (80)
  [Stream index: 0]
  Sequence number: 1 (relative sequence number)
  Acknowledgement number: 1 (relative ack number)
  Header length: 32 bytes
  ► Flags: 0x10 (ACK)
    Window size: 5888 (scaled)
  ► Checksum: 0x9529 [validation disabled]
  ► Options: (12 bytes)
  ► [SEQ/ACK analysis]
```

This is the final step in the process where our computer that started the 3 way handshake sets the ACK-bit and acknowledges the SYN from the other side.

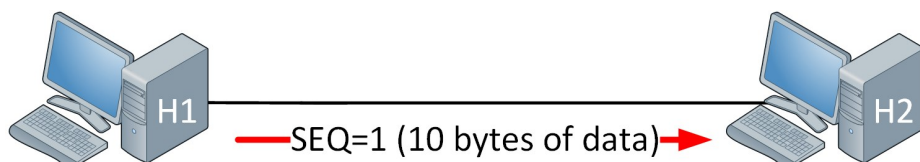
Are you following me so far? If you want to play a bit just start up Wireshark and see if you can capture a 3 way handshake yourself on your computer. Take a look at the different TCP packets and see if you can find the SYN, SYN-ACK and ACK's. Also check the different sequence numbers and see if you can find a pattern.

Phew so we have setup a connection using the 3 way handshake! Now we can start sending data...what else does TCP offer us? One of the things is **“flow control”**.

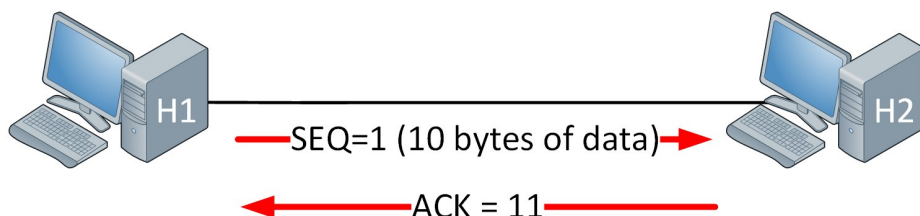
Imagine you have a fast computer transmitting data to a smartphone, obviously the computer could overburden the smartphone with traffic which is why we have flow control. In each TCP segment the receiver can specify in the “receive window” field how much data in bytes it wants to receive.

Our sending computer can only send data up to this size so the smartphone doesn’t get overburdened. The more data you can send each time the higher your throughput will be.

Let’s look at an example of how this all fits together:



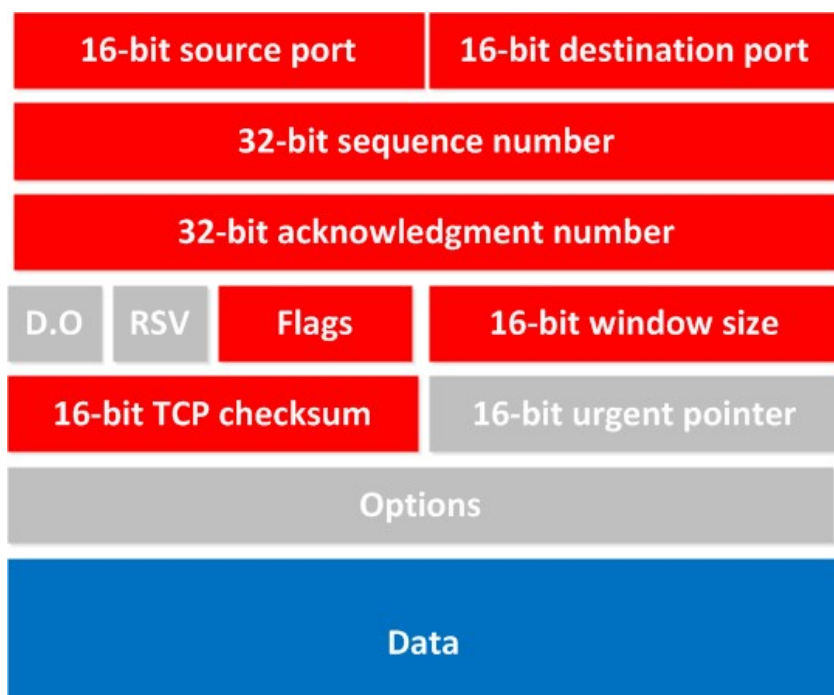
H1 has setup a connection with H2 by using the 3 way handshake. We are sending 10 bytes of Data which means our “window size” is 10 bytes. The sequence number is 1.



H2 is going to respond by sending “ACK=11” which means “thanks I received your 10 bytes, now send me #11 and the rest”. TCP is a reliable protocol which is why we have to acknowledge everything we are receiving.

The larger your window size, the higher your throughput will be. This makes sense because you are sending fewer ACK’s compared to the data you are sending.

TCP is a fairly complex protocol and if we look at the header you’ll see it has a lot more fields than UDP has:



The fields in gray are not important for us; everything in red is what I would like to tell you about.

As you can see there's a 16-bit source and destination port, **port numbers are used to determine for which application** this data is meant (This is how we go from the transport layer up to the higher layers in the OSI model).

You can see we have 32-bits that are used for our sequence numbers, and there's also 32-bits for the acknowledgment (ACK) reserved.

The "Flags" field is where TCP sets the different message types like "SYN" or "ACK".

Window size has a 16-bit field which specifies how many bytes of data you will send before you want an acknowledgment from the other side.

Finally there's a checksum and of course our data, the stuff we are actually trying to send to the other side.

Let's sum up what we have learned about TCP:

- It's a reliable protocol.
- Before you send data you will setup the connection by using the 3 way handshake.
- After sending X amount of bytes you will receive an acknowledgment (ACK) from the other side.
- How many bytes you send before you get an ACK is controlled by using the "window size".
- TCP can do retransmissions.

That's the end of this introduction. I hope you now have a better understanding of how TCP and UDP work and their differences. I can highly recommend to boot up Wireshark and to take a look at some TCP and UDP packets yourself. If you enjoyed this lesson please leave a comment or share it with your friends on Facebook or Twitter!

« **Previous Lesson**
Address Resolution Protocol
(ARP)

Next Lesson »
TCP Header

Tags: TCP

Forum Replies



dalittles77

Thank you for your videos! I know if I'm able to understand them then any/everybody should be able to understand because I have no computer background. Question, I would like to check out the Wireshark but am not sure as to how it works. Suggestions? Thanks again for your help.



Rene Molenaar

Hi Keith,

Glad to hear you like it! Before you try Wireshark, I would advise to dive into the OSI-model first (if you haven't done it yet). Using Wireshark is a nice exercise to see all the layers of the OSI-model.

It's best to download it and just give it a try:

<https://www.wireshark.org/download.html>

Make some captures when you try to visit a webpage or download something, see if you can recognize the TCP 3 way handshake and the different layers of the OSI-model.

If there's anything in particular you want, let me know and I'll create something ok?

Rene



gabrielba

Rene,

Excellent lesson!!! It's so good read a lesson like this, because you make it be so simple!

Thanks and Abraço



chrismchavez

Great lesson again! I like an application called tcpdump, which is like Wireshark but much less clunky. There is a little learning curve, but so much better for just watching traffic.



ReneMolenaar

Thanks 😊 Tcpdump is a great (linux) tool, it's really useful for capturing traffic and you don't need a GUI for it. It's very useful when you SSH into a remote box and want to capture some stuff.

For analysis I prefer Wireshark...you can capture with tcpdump and then look at it in Wireshark.

💬 **59 more replies!** Ask a question or join the discussion by visiting our [Community Forum](#)